

Adaptive HACT: Service-Matched Verification Evidence and Deployment Boundaries

Research manuscript*

September 21, 2026 | Version 5.0

Abstract

Agentic systems need a trustworthy answer to “which declared checks hold on this exact candidate?” without letting aggregation, display, or learning choices alter what is accepted. Adaptive HACT separates immutable check identities and acceptance predicates from the layout that aggregates their completion records: layouts are scored by full completion hyperedges rather than pairwise similarity, and generation fencing preserves conditional publication soundness across layout permutations. We make a learned-order compact balanced eight-ary tree the low-compilation-cost baseline, retain exact alphabetic dynamic programming as a reference, and relegate the slower, empirically inferior block compiler to an ablation. A fresh frozen-seed study of 27 train/validation/hold-out workloads shows hold-out padded-cost reductions of 23.43%, 10.75%, and 5.56% at 128, 512, and 1,024 checks on clustered completions, while independent and global controls show essentially no improvement—evidence that ordering pays only where completions cluster. Re-aggregating twelve archived source candidates under three layouts (36 layout replays) reduces whole-epoch compressed bytes by 4.36–41.84% versus original-order fixed8, yet remains larger than learned-order exact layouts; root-status inputs stay byte-identical across layouts. Rather than claiming that export savings cancel a different harness’s verification overhead, we derive an explicit codec- and service-matched deployment test—a fail-closed planner that outputs the minimum profitable copies under declared goodput and extra-cost assumptions. BPE and physical-WAN measurements remain unavailable; we supply actual-tokenizer admission and authenticated TLS measurement tools without presenting local protocol tests as those missing experiments. The result identifies both a lower-cost implementation and a narrow, auditable operating region, not a universal agent or verification accelerator.

Keywords: bounded evidence; verification contracts; bandwidth-limited logging; completion hyperedges; cost-aware layout selection.

1 Introduction

Many-agent systems need a reliable answer to “which declared checks hold on this exact candidate?” The answer must survive three independent pressures: per-check evidence volume grows with the registry; the evidence must travel over constrained links to supervisors, loggers, and reviewers; and whatever compresses or reorganizes that evidence must never change what is accepted. The agents’ task graph and conversational organization need not be the layout that aggregates this evidence. HACT separates the two: immutable check identities determine the acceptance contract, while a bounded-packet tree represents intermediate results. A smaller tree transcript is not automatically a faster build, a better agent, or a smaller root-status prompt.

The distinction matters economically. Historical source timings were 39.69 versus 39.77 seconds for fixed and learned layouts. A separate small harness paid roughly five seconds per episode for strict verification compared with direct pytest. Meanwhile, a historical compressed loopback export saved approximately 2.21 seconds *across twelve source streams*. These different cohorts cannot be combined into a measured end-to-end speedup, and neither observation justifies adding a strict wrapper solely to save transmission time.

*Author names and affiliations require confirmation before submission. Mathematical development, implementation, experiments, and writing were AI-assisted. This is a research artifact, not an accepted conference paper.

Three deployment decisions, not one. A trusted root-status consumer needs neither a learned tree nor all intermediate packets. A deployment that *already requires* protected verification and hierarchical evidence can compare fixed and learned layouts at marginal cost under that unchanged service. Adopting protected verification where ordinary pytest was sufficient is a different decision: its additional preparation and checking costs must be justified independently. We make this service distinction explicit in both the paper and a fail-closed planning API.

The simpler compiler is now primary. The prior block compiler was 1.54–5.34% more expensive in held-out bytes than a compact balanced eight-ary tree on the same learned order. We therefore remove block DP from the practical path. The new default catalogue retains the incumbent and proposes a balanced tree on a training-learned average-linkage order; separate validation compares full-wave costs. Exact DP remains a useful conditional optimum and reference. It can still produce smaller evidence than the practical default, as our source replays show. Low fitting cost and lowest byte cost are different objectives.

Contributions and evidence boundaries. The substantive research contribution is hyperedge-scored, identity-preserving verification aggregation and its conditional guarantees. Concretely, this paper contributes:

1. **A separation principle for verification evidence.** Immutable check identities and acceptance predicates are separated from the aggregation layout, so display and learning choices cannot alter what is accepted (Section 3; Theorem 4). A parity construction (Proposition 1) shows why pairwise similarity is insufficient for the full-wave objective.
2. **A practical no-DP compiler with evidence-matched selection.** The practical catalogue uses average-linkage ordering plus a balanced eight-ary tree, retains the incumbent, and requires an explicit priced improvement before any switch (Section 4, 5). The slower block compiler, empirically 1.54–5.34% worse in held-out bytes, moves to an ablation.
3. **Service- and codec-matched deployment accounting.** A fail-closed planning API rejects service mismatches and missing measurements, and turns assumed goodput and extra cost into an explicit minimum-profitable-copies test (Section 8; Table 4).
4. **A frozen, independently audited evaluation.** Twenty-seven fresh synthetic workloads, thirty-six archived-source layout replays and 131 audited summary comparisons quantify exactly where learning helps (clustered completions) and where it does not (independent and global controls) (Section 9; Figure 1).
5. **Honest boundary reporting.** BPE and physical-WAN measurements are absent; we supply actual-tokenizer admission and authenticated TLS measurement tools without presenting local protocol tests as those missing experiments.

Clustering, balanced trees, alphabetic DP, compression, break-even arithmetic and TLS are established techniques, not new inventions claimed here. The artifact is a tested research implementation; its tests do not establish production readiness or top-conference acceptance.

Roadmap. Section 3 fixes the trust boundary and cost model. Sections 4 and 5 develop the practical compiler and online selection. Section 6 states the permutation-stability theorem. Sections 8 and 7 describe service-matched delivery, admission and cost accounting. Section 9 reports the frozen evaluation; Section 11 states the deployment recommendation and remaining limits. Appendices retain tracking-regret analysis, the compiler ablation, all separation constructions, and the archived v3/v4 evaluations.

2 Related Work and Positioning

Testing, impact analysis, and incremental computation. Test prioritization seeks earlier fault exposure [1]; regression-test selection uses change dependencies [2, 3]. The earlier IC-HACT ranker uses controlled mutation outcomes to prioritize checks, an instance of established fault-exposure methods. It is held fixed in the new source comparisons. Self-adjusting computation and build-system research already formalize dependency-aware recomputation [4, 5]. In contrast to test selection, our strict pass requires the entire approved registry on the identified candidate. We do not reproduce NameRTS or Ekstazi and claim no superiority over them.

Seriation and authenticated layouts. Spectral seriation orders items using an affinity matrix [13]. Average-linkage clustering likewise provides an established proximity-based ordering. Generalized alphabetic-tree optimization supplies the subtree/forest recurrence used here [6]. Workload-adaptive authenticated structures, including Huffman–Merkle Trees [7], already optimize layouts under access patterns. Our price counts repeated completion-frontier updates, including strict-epoch construction, under a fixed packet cap. Pairwise affinity proposes an order; full hyperedge observations score it. No global optimality or unproved NP-hardness result is claimed for unrestricted joint ordering and tree synthesis.

Online decisions with switching costs. Fixed-share learning tracks changing experts [14]; dependent sampling reduces switches compared with independent weighted-majority draws [15]. Full-information and bandit switching-cost settings have different achievable regret guarantees [16]. We use the full-information setting because one recorded completion wave can be scored against every frozen certificate layout without reexecuting the tests. This is not full counterfactual feedback for arbitrary agent strategies: if a layout changes the tests’ execution schedule or model behavior, our exogenous-cost assumption may fail. Our bound specializes established tools to measured layout-installation costs; it is not a new minimax online-learning result.

Agent runtime assurance and evaluation. Proof-Carrying Agent Actions [8] and verification-portfolio commit gates [9] address related action-governance problems. Our local certificates are neither signatures nor zero-knowledge proofs. SWE-bench evaluates real repository issues and patches [17]; our source mutations and scripted I/O repairs must not be called SWE-bench results. The current integration measures real orchestration and verification, not model reasoning quality.

Logging and serialization controls. Telemetry systems already batch and compress records [18]. HACT is not a replacement for those techniques: fewer padded bytes need not imply fewer compressed bytes. We measure both. Moreover, flat result ledgers and a trusted root-status endpoint can be much smaller than an archive of all intermediate aggregation updates; they satisfy different delivery needs and must not be hidden from the comparison.

3 Model and Trust Boundary

3.1 Semantic registry, execution, and display

Let $\mathcal{R} = (g_0, \dots, g_{n-1})$ be a nonempty approved registry of unique check IDs. The candidate identity is

$$e = (\text{epoch}, \text{snapshot}, \text{checker}, \text{environment}, \text{registry}).$$

The snapshot covers source and protected acceptance inputs. The checker and environment are explicitly bound; an epoch increases even after an ABA return to identical source bytes. Only a trusted runner submits final PASS, FAIL, or UNKNOWN records. Missing, stale, skipped, incompatible, or timed-out outcomes are not passes.

An execution permutation σ_e determines when checks run. A layout $L = (\pi, T)$ independently maps tree positions to semantic registry indices. For a node covering positional interval $[i, j]$, its semantic coverage is

$$G_L(i, j) = \{g_{\pi(i)}, \dots, g_{\pi(j)}\}. \quad (1)$$

The stored layout must contain an exact permutation, and every internal node’s children must partition its positional interval without gaps or overlaps. The task graph is outside this mathematical tree; one worker can discharge many checks, and one integration check can cover several workers.

The acceptance predicate is independent of both permutations:

$$A(e) = \mathbf{1}\{\text{every } g_i \in \mathcal{R} \text{ has trusted, matching, current PASS evidence for } e\}. \quad (2)$$

An observed fail permits rejection before all checks finish. It does not assert coverage of the unexecuted remainder. Acceptance is relative to the approved predicates, not proof of all program behavior.

3.2 Packet and lifecycle costs

Each internal node of arity d emits a fixed-width packet

$$c(d) = H + Sd, \quad 2 \leq d \leq b = \lfloor (C - H)/S \rfloor. \quad (3)$$

Here $H = 512$, $S = 320$, and $C = 3072$ bytes, so $b = 8$. A mandatory field that exceeds its slot is rejected, not silently truncated. The cap applies to one certificate packet, not to all project memory, an entire registry manifest, or every LLM prompt. A manifest is a referenced control-plane artifact; its measured size is charged on migration.

Let $W_{e,1}, \dots, W_{e,q_e}$ be the semantic IDs whose final records become available between successive refreshes. For a strict new candidate epoch, the observed cost is

$$\ell_e(L) = W(L) + \sum_{a=1}^{q_e} \sum_{v \in \text{int}(T)} c(d_v) \mathbf{1}\{W_{e,a} \cap G_L(v) \neq \emptyset\}, \quad W(L) = \sum_v c(d_v). \quad (4)$$

The first term constructs the full initially unknown tree. The CLI batches an executed prefix into groups of eight, flushing its final partial group. A successful strict epoch runs the whole registry. This conservative implementation does not reuse a pass across distinct source snapshots.

We separately study persistent-state refresh traces that omit $W(L)$ at each tick, as in a cache with correctly specified incremental dependencies. Each topology installation is still cold and explicitly priced. These traces evaluate the controller’s cost model; they are not the strict candidate-gate timing experiment. They do not demonstrate safe dependency inference from arbitrary code.

3.3 Why more than invalidation or pairwise similarity is needed

The inherited HACT result shows that marginal check activation cannot identify the best aggregation tree under joint invalidation (Appendix C). IC-HACT further shows that even the full initial invalidation distribution cannot identify optimal progressive verification: early rejection and completion batching change the update sequence. Both distinctions remain relevant after allowing permutations.

Proposition 1 (Pairwise affinity is insufficient for full-wave scoring). *There exist two distributions with identical singleton and pairwise co-activity probabilities, hence identical Jaccard affinities, but different activation probabilities for the same three-check subtree.*

Proof. On IDs $\{0, 1, 2\}$, let P be uniform on $\emptyset, \{0, 1\}, \{0, 2\}, \{1, 2\}$ and Q uniform on $\{0\}, \{1\}, \{2\}, \{0, 1, 2\}$. Every singleton has probability $1/2$, every pair has joint probability $1/4$, and its Jaccard affinity is $1/3$. Yet the probability that the triple is hit is $3/4$ under P and 1 under Q . Thus a packet emitted on that interval has different expected cost. $\square$

This elementary parity construction is not a new fact about higher-order statistics. Its implication for the compiler is specific: a pairwise clustering objective cannot substitute for scoring recorded completion hyperedges. It does not, by itself, prove that these two distributions have different globally optimal layouts.

4 Learning Layouts Without Changing the Contract

4.1 Practical catalogue and full-wave selection

Training waves define counts N_i, N_{ij} and affinity

$$a_{ij} = \frac{N_{ij}}{N_i + N_j - N_{ij}}, \quad (5)$$

with zero for an unobserved union and zero diagonal. The practical proposal uses established average-linkage clustering: join the closest current clusters, choose endpoint orientation by cross-boundary affinity, and break ties canonically. All semantic IDs, including disconnected and unseen checks, remain included exactly once.

Given the learned order, group consecutive leaves into blocks of at most eight, promote a singleton unchanged, and repeat upward. This bottom-up compact balanced construction does not call a tree optimizer. The catalogue retains the incumbent first, includes original-order fixed8 when distinct, and proposes learned-order fixed8. Identical layouts are deduplicated. A separate validation cohort scores complete completion waves, including initial construction; ties retain the incumbent. The API returns an advisory choice, not deployment authorization. The registry and packet cap cannot change during selection.

For any proposal π , training episodes supply interval weights

$$r_\pi(i, j) = 1 + \frac{1}{m} \sum_{e=1}^m \sum_a \mathbf{1}\{W_{e,a} \cap G_L(i, j) \neq \emptyset\}. \quad (6)$$

These weights also define the exact-DP reference below. Pairwise affinity *proposes* an order; it does not replace the full-wave cost. The practical catalogue contains at most three candidates and uses no DP. Tree construction is $O(n)$ after ordering, not a linear-time claim for affinity construction, clustering or validation scoring. The reference implementation retains spectral ordering and exact fitting for research comparisons; neither is invoked by the practical default.

4.2 Exact conditional tree synthesis

Let $D(i, j)$ be the optimal subtree cost and $F_k(i, j)$ the optimal cost of an ordered forest with k nonempty children. With infeasible states equal to $+\infty$,

$$D(i, i) = 0, \quad (7)$$

$$F_1(i, j) = D(i, j), \quad (8)$$

$$F_k(i, j) = \min_{i+k-2 \leq u < j} \{F_{k-1}(i, u) + D(u+1, j)\}, \quad (9)$$

$$D(i, j) = \min_{2 \leq k \leq \min(b, j-i+1)} \{r_\pi(i, j)c(k) + F_k(i, j)\}. \quad (10)$$

Theorem 2 (Exactness conditional on order). *For a fixed π , nonnegative interval weights, and the packet model in Equation (3), Equation (10) returns a minimizing feasible tree in $O(bn^3)$ time and $O(bn^2)$ space.*

Proof. Every feasible root has an arity and a contiguous partition of its interval. Its local contribution depends only on that interval and arity; all remaining contributions belong to independent child subtrees. The forest recurrence enumerates every such partition. Induction on interval length proves exactness. There are $O(bn^2)$ states, each scanning at most n splits. The apparent $F_1 = D$ dependency is used only on shorter intervals when constructing a forest with at least two components. $\square$

This is a specialization of classical alphabetic-tree optimization [6]. The historical three-proposal reference multiplies compilation time by a constant, not by $n!$. Dense spectral computation can itself cost $O(n^3)$; the reference clustering implementation uses $O(n^2 \log n)$ heap operations and $O(n^2)$ storage.

Proposition 3 (Validation after learned ordering). *Condition on a training set that fixes K complete layouts. For m independent identically distributed validation epochs with each layout cost in $[0, B]$, let $\widehat{L}$ minimize empirical validation cost. With probability at least $1 - \delta$,*

$$J(\widehat{L}) \leq \min_{L \in \mathcal{C}} J(L) + 2B \sqrt{\frac{\log(2K/\delta)}{2m}}. \quad (11)$$

Proof. Conditionally, the catalogue is fixed. Hoeffding’s inequality [10] and a union bound over K layouts bound every empirical mean error by the displayed radius without its factor two. Inserting empirical costs on either side of the empirical minimizer yields the result. $\square$

The guarantee is only relative to the fixed catalogue. It neither justifies choosing among learned permutations on their training errors alone nor covers distribution shift. We do not report this potentially loose bound as an empirical confidence interval; reported intervals use independent experimental seeds.

4.3 Compiler identity and scope

The practical default has no hidden microtree optimization. Exact DP is a reference and optional offline candidate. The block-backbone proposition and implementation remain in Appendix B solely as an ablation; its restricted-class exactness does not override the observed simpler-baseline advantage. At 1,024 checks, order learning still takes seconds in our measurement, so layouts should be cached and compiled away from the agent’s per-action path. Validation of padded costs does not optimize compressed archives or model tokenization; adoption must measure the actual delivery representation.

5 Practical Online Selection: Price the Window, Not the Promise

Freeze a compiled catalogue $\mathcal{C} = \{L_1, \dots, L_K\}$. At event t , choose a layout before observing the current completion wave; afterwards score that same wave under every catalogue member. This is full-information feedback only because the check schedule is held fixed. It is not counterfactual feedback for arbitrary agent behavior.

A different layout is installed by serializing its canonical manifest and reconstructing its complete certificate tree:

$$M_{ij} = \begin{cases} 0, & L_i = L_j, \\ |\text{manifest}(L_j)| + W(L_j), & L_i \neq L_j. \end{cases} \quad \mathcal{L}_T = \sum_t \ell_t(A_t) + \sum_{t=2}^T M_{A_{t-1}, A_t}. \quad (12)$$

The initial installation is separate. These byte prices match the evaluated uncompressed migration protocol. A deployment using compression must measure its own priced wire representation; the DP objective is not automatically an optimum for a compressed log.

The primary practical rule. Let $\bar{\ell}_{t,i}$ be the mean cost on the last $\min(w, t-1)$ observed events, and $j_t = \arg \min_i \bar{\ell}_{t,i}$ with deterministic ties. Starting from the calibrated incumbent a , use the frozen v3 rule

$$A_t = \begin{cases} j_t, & w(\bar{\ell}_{t,a} - \bar{\ell}_{t,j_t}) > M_{a,j_t}, \\ a, & \text{otherwise,} \end{cases} \quad w = 64. \quad (13)$$

With no history, keep a . A tie in estimated savings does not trigger migration. During warm-up, the forecast horizon remains w even if fewer events were observed, matching the original baseline. This is an empirical

Algorithm 1 Deployable evidence control; inference and checking remain separate**Require:** Protected registry, checker and snapshot identities; frozen layout catalogue

```

1: for each event do
2:   Choose from past evidence using Equation (13), or keep the frozen layout
3:   if layout changes then
4:     Revoke tickets, increment generation, remap stable IDs, rebuild and price the installation
5:   end if
6:   Receive trusted completion records for the fixed check schedule
7:   Reject stale, conflicting or malformed records; aggregate bounded packets
8:   Publish only the complete current contract result; otherwise retain FAIL/UNKNOWN
9:   Expose a root-status card; retrieve diagnostic packets only when needed
10:  Score the observed event and append its cost vector to the bounded window
11: end for

```

break-even forecast, not proof that past savings repay a future switch. It can lose under abrupt reversal; preserving the frozen layout is always a legitimate alternative.

The runtime API enforces `choose()` before `observe()`, bounds history to 64 vectors, rejects nonfinite costs, and checkpoints only at event boundaries. Replaying all 80 archived v3 streams, with periodic checkpoint/resume, must reproduce the original baseline’s full action and byte-cost paths exactly. The exercise validates implementation equivalence, not a new performance dataset.

Why fixed-share is no longer primary. On v3’s stationary, long-shift and rapid-shift streams, the coupled fixed-share policy is more expensive than the priced window. Its standard tracking analysis therefore moves to Appendix A, with all negative results retained. We do not manufacture an adversarial workload merely to make that policy win. Neither the appendix bound nor the empirical window result establishes general superiority over other switching-cost policies.

6 Stable Evidence Through Permutation and Migration

A result token carries (epoch, g_i, b_i) , where b_i binds the semantic ID to the registry, source snapshot, checker, and environment. It does not carry a display position as its identity. A layout-specific kernel maps stable IDs to current positions. During migration it rebuilds from canonical trusted records; it never transplants an ordinal cache entry into a different permutation.

Publication tickets additionally carry a monotone layout generation and layout hash. A layout switch revokes the issued ticket, advances the generation, and forces reconstruction. Returning to an earlier tree therefore cannot revive that tree’s old ticket. An in-flight token may remain valid through a layout-only change, but a new candidate epoch fences it. Duplicate identical evidence is idempotent. Conflicting final records in one epoch revoke authorization: a recorded fail remains a fail; otherwise the obligation becomes unknown. Resolution requires a new epoch, not a more persuasive agent summary.

Theorem 4 (Layout-independent local publication soundness). *Assume complete approved predicates and dependency identities, a trusted checker executing immutable identified inputs, collision-resistant binding, and atomic local epoch/generation checks. For any sequence of valid layout permutations, any advisory learning outputs, and any delivered evidence order, authorization implies Equation (2). If each checker is sound for its declared predicate, those predicates hold on the authorized candidate.*

Proof. A canonical pass requires a current token for the same semantic ID and immutable identities. Permutation is a bijection, so rebuilding moves each such record to exactly one position without changing its meaning. Exact child partitioning preserves coverage and pass/fail/unknown counts by induction up the tree. The root can be authorized only with $(n, 0, 0)$ and matching current registry, epoch, generation, and

Table 1: Separate ledgers avoid converting byte savings into unsupported latency or token claims.

Measurement	How obtained	Included in optimized objective?
Certificate bytes	Actual padded packets; independently reconstructed from waves	Yes
Migration bytes	Canonical layout manifest plus forced full rebuild	Yes, online traces
Fit and scoring time	Monotonic-clock measurements	Reported separately
Preparation, copying, checking, aggregation	Wrapper timers and outer call duration	Reported separately
Active harness episode	Supervisor startup through task completion and shutdown	Reported separately
Prompt bytes and construction time	Captured actual HTTP messages and context builder	Reported; not tokens
Hosted token usage and API billing	No provider inference was run	Unknown, never zero-imputed

layout. Old tickets fail those checks. Conflicts prevent authorization. No choice made by the learner writes trusted outcomes or changes the registry. $\square$

This is a conditional local guarantee, not remote attestation or distributed consensus. An incomplete suite, compromised checker, omitted dependency, or nondeterministic external service can invalidate a broader correctness claim. The CLI’s file copy is not an OS sandbox. An external merge/deployment service must atomically compare the verified candidate and integration-head identities at commit; a report file alone is not that transaction.

7 Implementation and Cost Accounting

The v5 artifact adds `practical.py`, `deployment.py`, `token_budget.py` and `tls_probe.py`. The practical CLI produces a balanced catalogue without calling either exact or block DP. The historical exact and block compilers remain separately callable. The existing stable-ID guard, priced window, codec and Future Code bridge are retained. The strict CLI compares protected input *sets* and hashes, detecting added tests or configuration files as well as changed and deleted ones. Schema-2 contracts bind the approved checker and environment and refuse silent reuse of legacy contracts. Source manifests are checked before and after snapshotting and verification; mutation of an observed snapshot prevents a pass.

The bridge runs as an ordinary subprocess acceptance gate in the unmodified Future Code v1.2 runtime. The supervisor still owns worker leases, private attempts, bounded contexts, and serialized integration. Non-pass exits cause candidate rejection. Detailed evidence and binary certificate packets live outside candidate write scopes; only a bounded JSON result card returns to the worker. The plug-in accepts a precompiled layout. The online selector and migration guard are separately integrated and stress-tested as a library path; a live, self-retuning hosted deployment was not evaluated.

A different scalar objective could combine these quantities only with declared unit conversions and observable counterfactual costs. This implementation does not optimize unknown model latency or billing. The exact-byte claim also depends on the stated padded format; compression defines a different, potentially nonseparable objective for which this DP is not claimed exact.

8 Service-Matched Delivery and Admission

8.1 Do not conflate three consumers

The local aggregator receives bounded packets; a logging collector may receive their complete stream; a supervisor normally receives a status projection and retrieves details selectively. Define $\psi(e, c)$ as canonical status text containing semantic identity, counts and local authorization, but no display positions. For the same trusted semantic report,

$$\psi_{L_1}(e, c) = \psi_{L_2}(e, c). \quad (14)$$

Its byte length and token count under any fixed deterministic tokenizer are therefore identical. This identity is not a new theorem about tokenization. It supplies a negative control: layout savings cannot reduce a layout-independent root input. The card is a projection of a trusted report, not authority to accept arbitrary worker JSON.

Diagnostics preserve full header/card records and page whole packets under a 16,384-byte limit. A packet that cannot fit is rejected, not truncated. For a *real supplied tokenizer* τ and explicitly reserved framing budget r , the new admission API additionally requires

$$|p|_{\text{UTF8}} \leq C_{\text{bytes}}, \quad |\tau(\text{decode}_{\text{UTF8}}(p))| + r \leq C_{\text{tokens}}. \quad (15)$$

The implementation retokenizes each full proposed page, including its prefix. Token counts are not assumed additive across concatenations. Greedy packing guarantees the tested limits, not a minimum number of pages. Toy counters in API tests are not BPE measurements. The optional measurement command requires actual `tiktoken` software and vocabulary [19], fingerprints the encoding, and records per-page content hashes. Missing encodings produce UNKNOWN and null counts; there is no byte-ratio fallback. Provider framing and billed usage require a separately measured adapter, not a guessed constant.

8.2 Encoding and the simplest sufficient service

Canonical unpadding JSON, independently compressed packets, and whole-epoch DEFLATE preserve their input records with different operational boundaries. Whole-epoch compression requires the completed archive, whereas per-packet compression permits incremental export. The padded objective is not an exact optimizer of either compressed bytes or tokens. Practical selection is advisory until its gain is measured with the deployed codec.

For a trusted consumer needing only final status, a root endpoint or flat ledger can be much cheaper than the intermediate stream. This changes the diagnostic service, not necessarily the underlying check set. Comparing that smaller service with an exhaustive stream cannot demonstrate that a tree optimizer is efficient or necessary. The strict checker, data representation, and delivery demand must each be named explicitly.

8.3 Adoption cost is not marginal layout cost

Consider q copies of the *same* workload and delivery service through one nonoverlapped bottleneck. Let B_0, B_1 be codec-matched application bytes per copy, $R > 0$ effective application goodput, $E \geq 0$ additional encoding/decoding work per copy, and $C \geq 0$ consistently amortized one-time incremental work. Common setup cancels only when actually shared. The conditional net benefit is

$$d = (B_0 - B_1)/R - E, \quad \Delta(q) = qd - C. \quad (16)$$

A strictly profitable integer copy count exists only when $d > 0$, and then

$$q_{\min} = \lfloor C/d \rfloor + 1. \quad (17)$$

This is accounting, not a new networking or regret theorem. A zero gain retains the baseline. The implementation uses decimal arithmetic at equality and rejects mismatched service IDs. Missing C , E or R returns UNKNOWN, not zero. An explicit zero is permissible only as a declared assumption or measurement.

When both variants already use the same protected gate, C is *extra layout work*, not the full strict-wrapper cost. When adopting strict verification from ordinary testing, its additional cost must be justified independently; the services differ and the planner returns INCOMPARABLE. Repeating exports amortizes one candidate’s preparation, not the preparation of future new candidates. Parallel links, computation overlap, cached diagnostics, queueing and different connection policies require direct critical-path measurement. High RTT alone does not make byte savings larger if both exports pay the same setup and request count.

8.4 Remote measurement and security boundaries

The historical transfer experiment is receiver-paced loopback, not a WAN. The new opt-in measurement kit uses verified TLS server certificates and hostname checking; upload and nonlocal bind require explicit switches. Frames and decompressed archives are bounded, and acknowledgments check received length and digest. Local tests exercise cold and reused TLS sessions, rejected hostnames, untrusted roots, malformed acknowledgments and truncated records.

These tests validate a protocol path only. They provide no physical-WAN throughput, jitter, packet-loss or retransmission measurement. The supplied protocol records host and path provenance, codec, connection policy and matched payloads, and requests separately observed RTT/goodput and transport diagnostics where available, consistent with the distinctions in TCP throughput testing [21]. Fields that the minimal client cannot observe remain null. There is no receiver-side artificial pacing in a qualifying WAN trial. No public upload, account creation or credential search was performed.

TLS authenticates the configured server, not the checker’s truthfulness. Protected reference data, immutable candidate identity and local monotone state remain required for Theorem 4. There is no remote attestation, Byzantine consensus, client authorization service, non-repudiation or OS sandbox. Replacing both evidence and its trusted reference still defeats a digest-only comparison. The receiver is a bounded measurement utility to use behind an operator-controlled firewall, not a production ingestion service.

9 Evaluation of the Practical Revision

9.1 Provenance and frozen questions

The v5 contract was saved before the new runs. It fixes three sizes (128, 512, 1,024), three completion families (clustered, independent, global), and three seeds (55011–55013). Each of the 27 workloads has 128 training epochs, 128 validation epochs and 256 held-out epochs. Random draws are sequential disjoint samples; deterministic global observations naturally coincide. The catalogue is fit only on training data, selected on validation, and reported on held-out data. Cold strict-epoch construction is included in every objective. The nine size/family summaries use seed means and observed ranges, not episode-pooled significance tests.

A separate study re-aggregates twelve archived source candidates under three layouts, producing 36 layout replays. All use the same original-order trusted check records, the same executed sequence and batches of eight. Average-linkage orders were learned on the earlier training cohort and frozen before these candidates. None of these replays is a fresh checker execution, independent code repair, or new agent trial. Historical v3/v4 data remain in Appendices G and H, with their original cohort identities.

9.2 Practical default without exact or block DP

On clustered completions, mean held-out reductions are 23.43%, 10.75% and 5.56% as registry size increases (Figure 1). Global updates give no reduction; independent updates give zero except for one of the

Table 2: New v5 synthetic study. Reduction is selected-layout held-out padded cost relative to the original-order balanced incumbent; positive is better. Fit includes affinity/order learning and catalogue construction, not validation scoring. Three seeds per row; ranges are descriptive.

Checks	Family	Mean (%)	Range (%)	Fit median (s)	Changed
128	cluster	23.43	22.40–24.45	0.018	3/3
128	independent	0.00	0.00–0.00	0.016	0/3
128	global	0.00	0.00–0.00	0.032	0/3
512	cluster	10.75	10.06–11.13	0.523	3/3
512	independent	0.07	0.00–0.20	0.548	1/3
512	global	0.00	0.00–0.00	0.645	0/3
1,024	cluster	5.56	5.16–5.80	2.991	3/3
1,024	independent	0.00	0.00–0.00	3.086	1/3
1,024	global	0.00	0.00–0.00	3.462	0/3

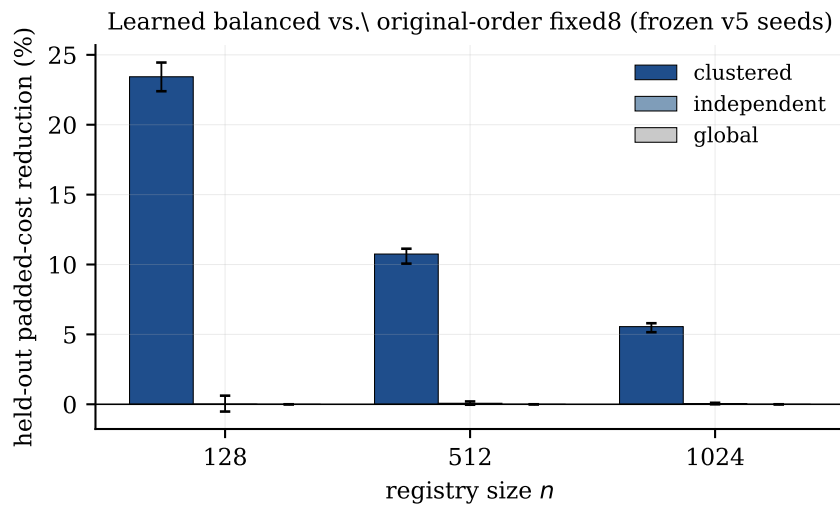


Figure 1: Held-out padded-cost reduction of the learned balanced eight-ary layout against original-order fixed8 across the frozen v5 seeds (three per cell). Bars show the seed mean; whiskers span the observed seed range. Learning pays only on clustered completions; independent and global controls show no systematic reduction.

three 512-check seeds, whose small gain produces a 0.07% mean. These controls do not support universal value from ordering. The selected method includes no DP, and a regression test makes any call to the exact fitting helper fail on this path.

The full catalogue fit median at 1,024 checks remains about 3 seconds; order learning is not free. The fixed construction can reuse a previously approved order in linear work, but no linear-time claim is made for the full learned pipeline. The smaller percentage at larger registries is consistent with substantial layout-independent cold construction in this benchmark; it should not be interpreted as a learned scaling law.

An initial execution exceeded its 120-second tool limit after saving 26 complete records. No process remained; explicit resume skipped those exact records and completed the remaining frozen seed without changing data or selection. All 27 observations and the interruption record are retained.

9.3 Source-evidence replays: simplicity is not byte optimality

Learned balanced layouts reduce whole-epoch compressed bytes by 41.84% for NetworkX [11], 4.36% for Toolz [12] and 18.53% for Future Code. These are *not* the larger v4 exact-layout savings relabeled as results of the new default. The balanced representation remains larger than learned exact in every project here.

Table 3: New serialization of twelve archived source candidates, four per project. The first three columns compare learned-order balanced8 with original-order fixed8. The last column compares learned balanced with learned exact after epoch compression: positive is a penalty. All include the codec’s application framing.

Project	Padded saving (%)	Canonical saving (%)	Epoch saving (%)	vs. exact penalty (%)
networkx	48.63	48.74	41.84	11.68
toolz	8.78	8.70	4.36	1.52
future	29.12	29.23	18.53	6.67

Table 4: Conditional minimum profitable copies of the *same twelve-candidate cohort* with whole-epoch compression. $E = 0$ is an assumption. The columns assume extra cost per candidate, paid once; cohort costs are 0.6, 6 and 60 seconds. These are not measured strict-wrapper overheads.

Assumed goodput	0.05 s/candidate	0.5 s/candidate	5 s/candidate
8 KiB/s	1	1	5
64 KiB/s	1	4	34
1 MiB/s	6	54	538

Exact fitting may therefore be appropriate when amortized delivery justifies its additional compilation; the practical default trades that potential gain for simpler fitting.

Across the twelve cases, batch-compressed application bytes are 416,046 for fixed8, 299,081 for learned balanced and 278,831 for the re-serialized learned exact layout. Semantic counts, verdicts and local authorization agree across all 36 replays. Their provenance binds the archived check records by digest. Serialization includes newly bound layout/run metadata; numerical bytes need not exactly reproduce a prior archive’s compressibility. The independent audit checks actual emitted bytes rather than substituting earlier totals.

Root-status text is identical in all twelve triples and occupies 593–594 UTF-8 bytes. Exhaustive diagnostic pages total 148 for fixed8, 95 for learned balanced and 87 for learned exact. The new default therefore uses 35.81% fewer pages than fixed8, not v4’s 41.22% exact-layout reduction. The largest emitted page is 16,364 bytes under a 16,384-byte limit. Reading every page is a specified exhaustive diagnostic service, not observed LLM behavior. There is no measured token reduction; root token equality follows from identical text for a fixed tokenizer, but no numerical BPE counts are asserted.

9.4 Codec-matched deployment scenarios, not measured speedups

The measured fixed-to-balanced compressed byte difference is 116,965 per twelve-candidate cohort. Equation (16) gives 1.784744 seconds of gross delivery saving per cohort copy at an *assumed* 64 KiB/s goodput, with $E = 0$. This is arithmetic from replayed byte sizes, not the older 2.21-second measured loopback saving of a different layout.

The 5-second column is a deliberately declared sensitivity scenario, not an estimate transferred from the historical harness. At 64 KiB/s and one export per candidate, the assumed 60 seconds of cohort overhead overwhelms the approximately 1.785-second delivery saving. Repeated delivery might change the equation, but the application must actually require it. With new candidates on each repetition, preparation recurs and cannot be amortized as the same C . All 27 scenarios (three codecs, three rates, three overhead assumptions) are available, including negative outcomes; Figure 2 visualizes all three codecs. The planner rejects mismatched services and missing measurements rather than inventing a benefit.

Minimum profitable copies of one 12-candidate cohort (v5 planning scenarios)

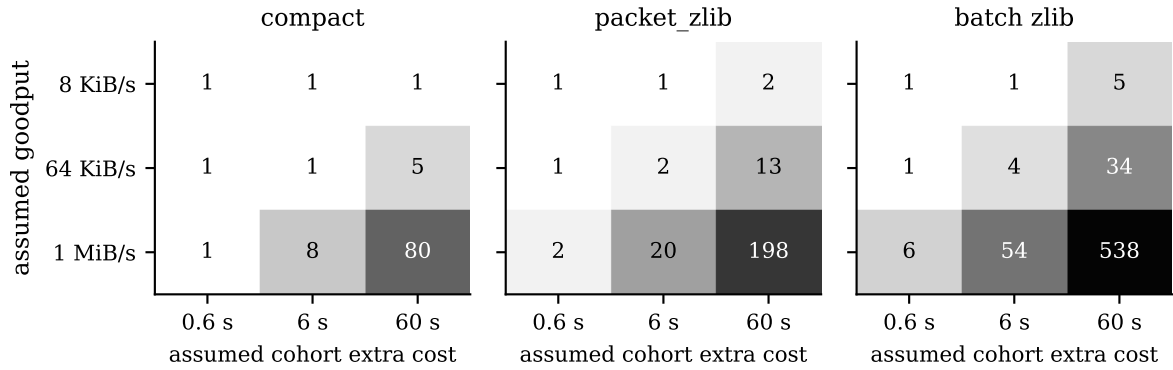


Figure 2: Minimum profitable copies of one twelve-candidate cohort for each codec, goodput and assumed extra cost (all 27 planning scenarios; darker means more copies are needed before learning pays). Table 4 gives the batch-zlib panel numerically. Adoption is plausible at low goodput with small overhead; at high goodput or large overhead, the unchanged incumbent is the rational choice. These are declared sensitivity scenarios, not measurements.

9.5 Historical controls remain controls

The earlier 288 loopback transfers belong to v4, use application pacing rather than real WAN conditions, and are not rerun or relabeled as v5 trials. For twelve compressed fixed/exact exports at 64 KiB/s, their measured totals were 6.624 and 4.413 seconds. The larger uncompressed totals were 33.499 and 18.954 seconds. Compression of fixed8 was more impactful than selecting an uncompressed learned layout. None of those durations is an IP/TLS throughput measurement or predicts a loss-prone path.

The original 39.69/39.77-second checker totals and 12.98/18.11/18.12-second median direct/fixed/learned harness durations remain negative controls. They came from different source/harness fixtures, so this paper does not subtract their times across cohorts. The historical window-controller and block-compiler evidence is retained unchanged. Block DP’s 1.54–5.34% penalty to same-order balanced8 is the reason it is now an ablation, not a contribution advertised as deployment acceleration.

9.6 Unmeasured endpoints and tested interfaces

Both requested BPE encodings, c1100k_base and o200k_base, remain UNKNOWN: the software was absent, package/vocabulary fetches failed, and a connector-retrieved build artifact was expired. No substitute tokenizer, model alias or byte division was used. The measurement tool records exact content-only counts when a real encoding is available; its unit tests use explicitly labeled toy counters only to validate admission logic.

No two authorized remote measurement hosts or designated live model endpoint were available. Physical-WAN and hosted-LLM trial counts are zero. The new TLS utility’s local compatibility tests check transport integrity, certificate rejection and bounded framing; they do not measure WAN jitter/loss or close the agent-evaluation gap. A reproducible deployment protocol is supplied, rather than a claimed remote experiment.

9.7 Software and independent audit

The v5 prototype passes 329 tests, including 76 new cases; the existing version-equality assertion is updated to require 5.0.0 consistently in runtime and package metadata. A duplicate test-module basename and the initially inconsistent version were corrected without weakening behavior checks; failure logs are retained. New tests cover no-DP compilation, service mismatch, missing costs, exact break-even, whole-text token

admission, TLS trust failures and deliberate data tampering.

A separate auditor imports no producer optimizer, selector or codec. It reconstructs full-initialization and wave costs for all 27 workloads, checks 36 serialized replays against original evidence digests and verdicts, reconstructs all codec byte sizes and diagnostic pages, and independently checks 27 planning scenarios and 131 summary comparisons. These checks validate recorded arithmetic and provenance, not third-party security certification. Release verification separately records extracted-source tests, the unchanged Future Code suite, installed-source hashes and archive integrity.

10 Deployment Recommendation and Remaining Limits

Choose a service before an algorithm. Do not deploy the strict wrapper solely on the strength of these transmission savings. For local trusted status, use the root projection; for final per-test states, a flat ledger may suffice. Where protected hierarchical evidence is already required, enable a suitable codec, retain the incumbent, and consider learned-order balanced fixed8 as the low-fitting-cost candidate. Exact DP remains an offline reference and can be worthwhile when its extra byte reduction pays back. Block DP is an ablation, not the default.

A cost calculator is not measured payback. The new scenarios expose when assumed extra costs dominate compressed delivery savings. They neither import a five-second overhead from a different fixture as though paired with each source case nor assume unlimited repeat exports. Dynamic adaptation is optional; the priced window selects already compiled candidates and can regress under rapid change. Whole-epoch compression changes streaming latency, and high-latency links do not necessarily favor HACT. Those are deployment questions, not consequences of the tree objective.

New default, new evidence boundary. The practical recommendation changed after the v4 review. It is therefore tested on new frozen synthetic seeds with disjoint train/validation/hold-out draws. The twelve source candidates are reused diagnostic evidence and are not an independent test of model-generated repairs. On these replays, learned balanced trees are smaller than original-order fixed8 yet larger than exact learned layouts. The recommendation is about compilation simplicity, not universal superiority. Three seeds per setting support descriptive ranges, not precise generalization claims.

Unclosed empirical requests. Neither a physical WAN nor actual public-vocabulary BPE was evaluated in this release. Software/vocabulary retrieval failed, and no authorized pair of remote controlled hosts was available. The optional tools and local tests do not close these experimental gaps. No hosted model was called, and there is no new SWE-bench or natural historical-bug cohort. The actual SWE-bench harness uses isolated repository environments [20]; scripted source mutations are not equivalent. Source-level token counts, when measured later, will still not establish provider billing or better agent reasoning.

Trust and scope. Future Code remains a single-host supervisor. The guard and TLS utility are not a distributed consensus protocol or fault-tolerant deployment. A passing declared suite cannot establish absence of arbitrary defects. Authorization requires an uncompromised checker, correctly scoped inputs and protected reference state. A separate integration service must bind the verified candidate and integration head atomically; no PDF table or root JSON card substitutes for that step.

11 Conclusion

Adaptive HACT preserves an acceptance contract while changing the organization of its evidence. Full completion hyperedges supply the cost signal; stable IDs and generation fencing preserve local meaning.

The practical path now uses a balanced tree over a learned order, retains the incumbent and performs no DP. New held-out synthetic tests and archived-source replays support a limited byte-efficiency claim, not universal latency gains. Service-matched accounting makes the adoption boundary explicit, while the missing physical-WAN, BPE and live-agent experiments remain identified rather than replaced by proxies. The result is a simpler implementation and a more falsifiable deployment claim.

References

- [1] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001.
<https://doi.org/10.1109/32.962562>.
- [2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015.
<https://users.ece.utexas.edu/~gligoric/papers/GligoricETAL15Ekstazi.pdf>.
- [3] You Wang, Michael Pradel, and Zhongxin Liu. Names Are All You Need: Effective and Safe Regression Test Selection for Python. arXiv:2605.25356v1, May 2026.
<https://arxiv.org/abs/2605.25356v1>.
- [4] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006.
<https://doi.org/10.1016/j.entcs.2005.11.043>.
- [5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018.
<https://doi.org/10.1145/3236774>.
- [6] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Author-hosted manuscript, Sections 4.1–4.4; date not established from the retrieved manuscript.
<https://www.mpedram.com/Papers/alp-trees-journal.pdf>.
- [7] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026.
<https://arxiv.org/abs/2608.25206v1>.
- [8] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104v1, June 2026.
<https://arxiv.org/abs/2606.04104v1>.
- [9] Zihao Zheng, Baichuan Li, Junyi Yao, and Jiayu Long. Engineering Reliable Commit Gates for Agentic AI: Cost-Aware Verification Portfolios under Common-Mode Data Failures. arXiv:2609.10969v1, September 2026.
<https://arxiv.org/abs/2609.10969v1>.
- [10] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
<https://doi.org/10.1080/01621459.1963.10500830>.
- [11] NetworkX developers. NetworkX 3.6.1 release documentation and distributed source tests.
https://networkx.org/documentation/stable/release/release_3.6.1.html.
- [12] PyToolz contributors. Toolz API documentation and distributed source tests.
<https://toolz.readthedocs.io/en/stable/api.html>.

- [13] Jonathan E. Atkins, Erik G. Boman, and Bruce Hendrickson. A Spectral Algorithm for Seriation and the Consecutive Ones Problem. *SIAM Journal on Computing*, 28(1):297–310, 1998.
<https://doi.org/10.1137/S0097539795285771>.
- [14] Mark Herbster and Manfred K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32:151–178, 1998.
<https://doi.org/10.1023/A:1007424614876>.
- [15] Sascha Geulen, Berthold Vöcking, and Melanie Winkler. Regret Minimization for Online Buffering Problems Using the Weighted Majority Algorithm. *Proceedings of COLT*, pages 132–143, 2010.
<https://www.learningtheory.org/colt2010/conference-website/papers/105geulen.pdf>.
- [16] Nicolò Cesa-Bianchi, Ofer Dekel, and Ohad Shamir. Online Learning with Switching Costs and Other Adaptive Adversaries. *Advances in Neural Information Processing Systems*, 26, 2013.
<https://papers.neurips.cc/paper/5151-online-learning-with-switching-costs-and-other-adaptive-adversaries.pdf>.
- [17] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024.
<https://arxiv.org/abs/2310.06770>.
- [18] OpenTelemetry contributors. Batch Processor documentation, official collector implementation. Accessed September 20, 2026.
<https://github.com/open-telemetry/opentelemetry-collector/tree/main/processor/batchprocessor>.
- [19] OpenAI contributors. tiktoken: BPE tokenization implementation and encoding registry. Accessed September 21, 2026; package/vocabulary unavailable in the v5 measurement runtime.
<https://github.com/openai/tiktoken>.
- [20] SWE-bench contributors. The Harness: Docker-based patch evaluation reference. Accessed September 20, 2026.
<https://www.swebench.com/SWE-bench/reference/harness/>.
- [21] B. Constantine, G. Forget, R. Geib, and R. Schrage. Framework for TCP Throughput Testing. RFC 6349, August 2011.
<https://www.rfc-editor.org/rfc/rfc6349.html>.

A Optional Fixed-Share Analysis: Not the Deployment Recommendation

This section retains v3’s former Theorem 4 and its assumptions. It is an optional comparator, not the practical selector recommended in v4. On the measured streams it loses to the deterministic priced-window rule in three regimes. We have not searched for a favorable adversarial stream after seeing those results, and make no claim that a tracking bound implies an empirical advantage.

A.1 Fixed-share weights with maximal coupling

Let a declared bound R cover each round’s loss spread, $\max_i \ell_t(i) - \min_j \ell_t(j) \leq R$. Reject a measurement exceeding it instead of clipping the cost. Define $z_{t,i} = (\ell_t(i) - \min_j \ell_t(j))/R$. Starting at $p_1(i) = 1/K$,

update

$$q_{t+1}(i) = \frac{p_t(i) \exp(-\eta z_{t,i})}{\sum_j p_t(j) \exp(-\eta z_{t,j})}, \quad (18)$$

$$p_{t+1}(i) = (1 - \alpha)q_{t+1}(i) + \alpha/K, \quad 0 < \eta \leq 1, \quad 0 \leq \alpha < 1. \quad (19)$$

Independent draws from each new distribution can waste migrations. Instead, conditional on $A_t = i$, retain i with probability $\min\{1, p_{t+1}(i)/p_t(i)\}$. If it is not retained, sample from the normalized positive residual $(p_{t+1} - p_t)_+$. This maximal coupling has marginal p_{t+1} and switch probability $\text{TV}(p_t, p_{t+1})$. Established dependent-sampling methods motivate this choice [15]; we do not claim it as a new sampling algorithm.

Theorem 5 (Catalogue tracking with migration). *Assume a frozen catalogue, a fixed exogenous sequence of full-information loss vectors, spread at most R , and $M_{ij} \leq M$. For any comparator path $u_{1:T}$ with s changes and $0 < \alpha < 1$, the coupled policy satisfies*

$$\mathbb{E}[\mathcal{L}_T] - \sum_t \ell_t(u_t) \leq R \left[\frac{\log K + s \log(K/\alpha) + (T - 1 - s) \log(1/(1 - \alpha))}{\eta} + \frac{\eta T}{8} \right] + M(T - 1)(\eta + \alpha). \quad (20)$$

For $\alpha = 0$ the stationary-comparator bound has numerator $\log K$ and applies only to $s = 0$.

Proof. The fixed-share update is exponential weighting over paths with an initial uniform prior, self-transition mass at least $1 - \alpha$, and transition mass at least α/K . The negative log prior of a path with s switches is bounded by the numerator. The bounded exponential-moment inequality gives expected normalized service regret at most that numerator divided by η , plus $\eta T/8$ (Appendix D). Since losses are exogenous, the weights are independent of the sampled action path, and the coupling preserves each unconditional marginal. Exponential reweighting changes total variation by at most η ; uniform mixing adds at most α . Thus the expected switch count is at most $(T - 1)(\eta + \alpha)$. Multiplication by M and addition of service regret proves the bound. $\square$

The bound also upper-bounds regret relative to the comparator’s service *plus* nonnegative migration cost, but can be quite loose. It is not a per-run improvement guarantee. Choosing fixed $\eta = .5$ and $\alpha = .01$, as in our frozen experiment, need not make this bound sublinear with increasing T . Horizon-dependent tuning yields sublinear stationary bounds (Appendix D), without creating a guarantee against arbitrary reactive agents or newly introduced catalogue entries.

B Block-Backbone Compiler: Retained Ablation

The exact compiler’s cubic cost makes frequent full recompilation inappropriate. For a supplied display order, divide the leaves into consecutive blocks of at most s leaves, and fix a compact b -ary macro-tree on those blocks. Optimize each block’s microtree using its restricted completion waves and strict-epoch initialization cost. Replace each macro leaf by its optimized microtree. No evaluation waves enter this process.

Proposition 6 (Exactness in the block-backbone class). *With display order, block boundaries and macro-tree fixed, independent exact optimization of the blocks minimizes the training objective over every layout in that restricted class. The dynamic-programming work is $O(bns^2)$ with $O(bs^2)$ reusable DP workspace, excluding order learning and wave preparation.*

Proof. Every macro node covers a fixed union of complete blocks, so its activation weight and packet price are constant across the class. All variable costs lie inside disjoint microtrees. The objective is a constant plus the sum of their independent objectives. Conditional exactness therefore gives the minimizing member. There are at most $\lceil n/s \rceil$ subproblems, each of size at most s , yielding the stated DP bound. Reusing the subproblem arrays yields the workspace bound. $\square$

For the reference implementation, M recorded waves and L total wave memberships add filtering and weight-preparation work of $O(\lceil n/s \rceil L + Mns)$, and storage for the input, output and local $M \times s$ arrays. This is not a linear-time claim for the entire pipeline. Dense affinity construction and order learning remain separate costs. No bounded approximation ratio against the unrestricted exact tree is asserted; held-out cost can deteriorate. The historical experiment froze $s = 32$ before measurement. Its negative result motivates excluding this compiler from the v5 practical path.

The original 18-workload v4 scaling data remain in the artifact. Block DP reduced fitting time relative to exact DP but lost to same-order balanced fixed8 in held-out bytes. Restricted-class exactness is a mathematical property, not an empirical recommendation.

C Information Separations Retained from Earlier Versions

These results concern the information available to a selector. They are not hardness results for all adaptive tree algorithms, and they do not imply that a complete hypergraph must be recovered: interval hit statistics are sufficient for a fixed order and objective.

C.1 Marginals do not determine a co-invalidation optimum

Consider four ordered leaves, binary trees, and a constant internal packet price $c > 0$. Let $\mathcal{D}_A$ choose $\{0, 1\}$ or $\{2, 3\}$ equiprobably and $\mathcal{D}_B$ choose $\{0, 3\}$ or $\{1, 2\}$ equiprobably. All leaf marginal activation probabilities equal $1/2$ in both distributions. The root and every three-leaf interval are always activated. The five tree shapes therefore fall into the following classes:

Ordered tree	J_A/c	J_B/c
$((0, 1), (2, 3))$	2	3
$(0, ((1, 2), 3)), ((0, (1, 2)), 3)$	3	2.5
$(0, (1, (2, 3))), (((0, 1), 2), 3)$	2.5	3

Optimal costs are $2c$ and $2.5c$. A deterministic marginal-only selector receives identical input and must choose the same class, incurring at least $c/2$ regret on one distribution. The last class is weakly dominated by the balanced tree. If a randomized selector chooses the balanced tree with probability q , its regrets are at least $(1 - q)c$ and $qc/2$. Their maximum is minimized at $q = 2/3$ and equals $c/3$, which is attainable for this instance. A fixed initialization price adds the same $3c$ to all binary shapes and does not alter the separation.

C.2 Joint invalidation does not determine progressive layout

Every epoch now invalidates all four checks, and execution order is $(0, 1, 2, 3)$ with singleton completion waves. In workload A , check 0 fails, so only its wave is executed. In workload B , check 3 fails, so all four checks execute. Initial invalidation is identical, even jointly.

Every binary tree has three internal nodes. Its cost in A is $(3 + \text{depth}(0))c$, minimized at $4c$ by a root-0 comb. Its cost in B is $(3 + \sum_i \text{depth}(i))c$, minimized at $11c$ by the balanced tree. The root-0 comb costs $12c$ in B , whereas the balanced tree costs $5c$ in A . The remaining shapes are weakly dominated. An invalidation-only randomized selector using a root-0 comb with probability p incurs regrets at least $(1 - p)c$ and pc , whose maximum is at least $c/2$. A deterministic selector incurs at least c . The example proves a separation between stale-set information and schedule-induced completion information; it does not prohibit a selector that observes both.

D Cost, Statistical, and Online Proof Details

D.1 Exact serialization accounting

On the first refresh in a strict epoch, every internal node emits one packet. Within a fixed tree and registry, the cache uses monotone descendant-generation stamps. A subsequent wave emits a node's packet exactly when at least one final record in its semantic coverage changed. Multiple completed descendants in one wave still induce one packet at that node. Summing initialization and wave contributions yields Equation (4). After migration, a new kernel remaps canonical IDs and the old ordinal cache is not reused; the entire cold rebuild is charged in Equation (12).

This argument concerns logical and actual serialized packet lengths. The separate v4 study measures controlled application-level export time, not production-network throughput. It does not include raw log streams, TLS framing, operating-system scheduling, or Python traversal costs. The reference kernel validates and traverses trees and is not claimed to have logarithmic total runtime. Small contexts do not imply small total system state.

D.2 Full-refresh lower bound and global negative control

For an n -leaf tree with m internal nodes, the number of edges is $n + m - 1$, so $\sum_v d_v = n + m - 1$. Therefore

$$W(L) = S(n - 1) + (H + S)m \geq S(n - 1) + (H + S) \left\lceil \frac{n - 1}{b - 1} \right\rceil. \quad (21)$$

The inequality follows from $n - 1 = \sum_v (d_v - 1) \leq m(b - 1)$. At $n = 64, b = 8$, a complete eight-child construction attains the bound with $m = 9$ and $W = 27,648$ bytes. Thus all selected global-control layouts can tie an analytically minimal full refresh. Strict initialization followed by one global completion wave costs $2W$. This explains the exact negative control without assuming every balanced tree at every n attains the lower bound.

D.3 Uniform statistical control at a fixed order

For one activation wave per episode and a fixed registry order, let $M_n = n(n + 1)/2$ and

$$\epsilon = \sqrt{\log(2M_n/\delta)/(2m)}.$$

The interval hit indicators are bounded independent variables *across* independent episodes, though checks within an episode may be arbitrarily dependent. Hoeffding plus a union bound gives simultaneous interval-frequency error at most ϵ . For every feasible tree, $|\hat{J}(T) - J(T)| \leq W(T)\epsilon$. Inserting the empirical minimizer gives regret at most $[W(\hat{T}) + W(T^*)]\epsilon \leq 2(n - 1)C\epsilon$. This is the original HACT bound, retained for attribution. It is not automatically valid uniformly over arbitrary data-selected permutations; the separate held-out catalogue bound in Proposition 3 addresses that selection step.

For strict epochs with at most q completion waves, a loose range is $B = (q + 1)(n - 1)C$. Under the evaluated sequential eight-record flush rule with at most one final record per obligation, $q \leq \lceil n/8 \rceil$. A narrower observed range is not automatically a proved bound for an unobserved deployment distribution.

D.4 Maximal coupling and its switch count

Let p, p' be consecutive distributions. The probability mass that remains at index i under the coupling is $\min(p_i, p'_i)$. The remaining mass is distributed as $(p' - p)_+$, so the new marginal is exactly p' and

$$\mathbb{P}(A_{t+1} \neq A_t) = 1 - \sum_i \min(p_i, p'_i) = \text{TV}(p, p').$$

For exponential reweighting by $a_i = \exp(-\eta z_i)$ with $z_i \in [0, 1]$, the normalizer is at most one, hence $q_i \geq e^{-\eta} p_i$. Thus $\sum_i \min(p_i, q_i) \geq e^{-\eta}$ and $\text{TV}(p, q) \leq 1 - e^{-\eta} \leq \eta$. Mixing q with the uniform distribution by weight α changes total variation by at most α . The triangle inequality gives the switch bound in Theorem 5. Zero old mass is never the probability of a selected previous action in exact arithmetic; the implementation rejects inconsistent checkpoints.

D.5 Fixed-share path argument

A path $u_{1:T}$ with s changes has prior mass at least

$$P(u_{1:T}) \geq K^{-1}(\alpha/K)^s(1-\alpha)^{T-1-s}.$$

The share update is the forward recursion for exponential weighting over such paths. If $Z_t = \sum_i p_t(i) \exp(-\eta z_{t,i})$, then the bounded exponential-moment inequality implies

$$\log Z_t \leq -\eta \langle p_t, z_t \rangle + \eta^2/8.$$

On the other hand, the cumulative mixture likelihood is at least the comparator path's prior times its exponential loss. Combining the inequalities yields

$$\sum_t \langle p_t, z_t \rangle - \sum_t z_{t,u_t} \leq -\log P(u_{1:T})/\eta + \eta T/8.$$

Centering losses by their per-round minimum cancels in regret, and multiplying by R restores byte units. For a fixed exogenous cost sequence, each p_t is determined by past full-information observations independently of the sampled action sequence; maximal coupling preserves its marginal. Therefore expected sampled service equals the left-side weighted service. Adding at most M per expected switch proves Equation (20).

The result does not cover a cost sequence chosen in reaction to the coupled hidden action history. It also does not cover agents changing future task choices because a layout changed the text they read. These limitations are why the paper distinguishes a verification-plane cost experiment from an end-to-end learning claim.

D.6 Horizon dependence and break-even interpretation

For $K \geq 2$, $s = 0$, and $\alpha = 0$, write $B_T = RT/8 + M(T-1)$. The bound is $R \log K/\eta + \eta B_T$. When $\sqrt{R \log K/B_T} \leq 1$, choosing that learning rate gives at most $2\sqrt{R \log K B_T}$ expected regret, which is sublinear for fixed R, M, K . This is an established style of switching-cost bound, not a new rate. With a known sparse switch budget, selecting a horizon-dependent share similarly prices tracking complexity. Our fixed-parameter experiments do not invoke such a tuned asymptotic guarantee.

A single empirical switch with estimated future saving d and anticipated remaining horizon h satisfies the familiar break-even inequality $hd > M_{ij}$. This is arithmetic, not a drift theorem: savings may reverse before h events. The included window rule uses its past 64 events as that estimate and is deliberately reported as a heuristic. All choices, including greedy choices that ignore migration when deciding, are charged their actual transition price during evaluation.

E Experimental Reproduction and Threats to Validity

E.1 Software and data identities

The archived v3 runs used Python 3.13.5 on Linux x86_64; exact installed package and platform records accompany emitted reports. Compilers were measured with `OPENBLAS_NUM_THREADS=1`. No accelerator is required. Vended upstream sources and tests are retained with their notices. Every fresh mutation is

checked against its archived base-source hash, replacement span, and resulting source hash. This binds a result to a specific intervention, not a natural historical bug distribution.

For the archived v3 record, there are three named output categories: `historical/results_v2` contains previous observations; `results/v3` contains v3's generated layouts, raw waves, online paths, fresh subprocess reports, and harness records; `paper/generated` contains tables regenerated from those files. Running reproduction in a fresh destination avoids overwriting the released record. The release manifest identifies the exact packaged files. The final extracted archive is tested again before delivery. New v4 transfers, compiler traces, contexts, summaries and audits are stored separately in `results/v4`; they do not change the number of earlier checker or harness executions. V5 adds `results/v5` for the practical catalogue, archived-source re-aggregation, conditional deployment accounting and failed BPE attempts. Neither a new layout nor a reproduced experiment is counted as another repaired candidate.

E.2 Costs not hidden by replay

Ordering measurements include strict initialization; online traces explicitly use persistent-state incremental service plus cold switch prices. These are different lifecycles. Source replay uses a fixed previous ranker; fresh paired executions hold that ranker constant. The I/O harness does not compare different model-generated solutions: it sends identical scripted patch content to the same real runtime under different gates. Its model endpoint produces no provider token-usage field, so absent usage remains null in the records. Prompt byte lengths are measured directly and never converted to tokens by a heuristic multiplier.

Timing rows report outer-wrapper or active-episode boundaries explicitly. The wrapper's internal total ends just before final report and packet-file writes; the fresh study additionally records the outer call duration including those writes. Fit, contract approval, and baseline preparation are setup costs, not silently amortized into a claimed end-to-end speedup. With only three harness repeats, ranges are more honest than a precise broad-population confidence claim.

E.3 Test adequacy, nondeterminism, and I/O

The fixture performs real disk and loopback I/O but is intentionally controlled. It is not a test of internet reliability, production SQLite contention, nondeterministic service responses, or all flaky-test policies. When a collected check is skipped, missing, or inconsistent, it is not accepted as a pass. Two contradictory final records in one epoch trigger conflict handling, and an operator must establish a new epoch rather than cherry-pick the favorable record. Actual source tests can themselves be incomplete; agreement with their oracle is only agreement with the declared acceptance contract.

F Artifact Usage and Integration Contract

A reviewed operator baseline is initialized outside candidate write scopes:

```
python -m ichact init --project /work/trusted-baseline \
  --contract /work/contracts/approved.json \
  --output /work/reports/baseline --test-path tests

python -m ichact verify --project /work/candidate \
  --contract /work/contracts/approved.json \
  --output /work/reports/candidate-001 \
  --layout /work/contracts/layout.json
```

A layout must match the full semantic registry. The optional execution-order file is separate and must also be an exact permutation, never a subset. Legacy schema-1 contracts must be reviewed and reinitialized; the software does not silently reinterpret them.

The Future Code gate invokes the installed module or a pinned absolute bridge path outside the candidate:

```
python -m ichact.future_bridge \
  --contract /work/contracts/approved.json \
  --reports /work/verification-reports \
  --layout /work/contracts/layout.json
```

The working directory is the candidate snapshot. Exit 0 means complete current pass; nonzero means rejection or unknown. This CLI does not itself deploy or merge. The original harness performs its own staged-source fingerprint and integration checks. For hostile code, use an outer container/VM and do not give it access to secrets or trusted contract storage.

The artifact contains an actual scripted harness campaign, a separate live-endpoint configuration guide, deterministic tests, an independent audit, and an isolated reproduction launcher. The live guide is a deployment recipe, not evidence that paid inference or hosted evaluation was performed.

G Retained v3 Evaluation: Historical Evidence

All results in this appendix were recorded in v3. “Fresh below distinguishes its actual checker executions from its v2 replay, not new v4 runs. No outcomes in this appendix are pooled with the v4 transport or v5 layout studies.

G.1 Questions, frozen protocols, and provenance

The evaluation asks four questions: Does learned display order recover savings lost to arbitrary registries? Can online selection pay for its own migrations under drift? Do results survive actual source checking? What overhead remains when the verifier is called by a real agent harness?

The v3 experimental contract fixes four synthetic families, seeds 0–19, training/validation/evaluation sizes, packet widths, catalogue policies, and online parameters before their runs. Ordering uses $n = 64$, 256 training, 128 validation, and 512 evaluation episodes per seed. Online evaluation uses 2,048 events per stream and $\eta = .5$, $\alpha = .01$. We retain these parameters after observing unfavorable results. All synthetic random seeds and realized waves are stored. Error bars are percentile bootstrap intervals over 20 paired *seed-level* measurements, not over thousands of correlated inner events; 10,000 bootstrap resamples use seed 94001. These are descriptive 95% intervals, not simultaneous guarantees over every comparison.

The source-study lineage is explicit. The artifact retains v2’s 121 source interventions: 45 training, 43 development, and 33 later confirmation cases. One development execution timed out and remains unknown. This revision reuses those outcome records for training, validation, and *retrospective replay*; it does not relabel the 33 published cases as new unseen v3 data. It then runs 24 fresh paired candidate checks on 12 selected archived interventions, plus trusted baselines and restored-source checks. Finally it adds nine actual runtime episodes on a new non-arithmetic I/O fixture. Repetition, provenance, and scope are different across these studies and are not pooled into a single success-rate statistic.

G.2 RQ1: Ordering robustness and negative controls

Shuffled-cluster workloads generate eight-check groups in a randomly permuted semantic registry, with a 12% probability of one additional check. Independent workloads activate each check with probability $8/n$. Singleton workloads use a Zipf-like exponent 1.2; global workloads activate all checks. Every reported ordering cost includes a fresh full-tree construction. The baseline “original DP” uses the same exact compiler and full-wave observations but preserves registry order; “fixed8” is the compact bottom-up eight-child tree. Selection uses only the separate validation cohort.

Table 5: Archived v3. Mean paired byte reduction in percent; positive is better. Brackets are seed-bootstrap 95% intervals. Each row uses 20 seeds and 512 held-out episodes per seed. Initialization is included.

Workload	Versus original-order DP	Versus fixed8
cluster	26.38 [25.94, 26.79]	28.46 [27.91, 29.02]
independent	-0.04 [-0.17, 0.09]	-0.04 [-0.17, 0.09]
singleton	0.00 [0.00, 0.00]	4.13 [4.02, 4.24]
global	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]

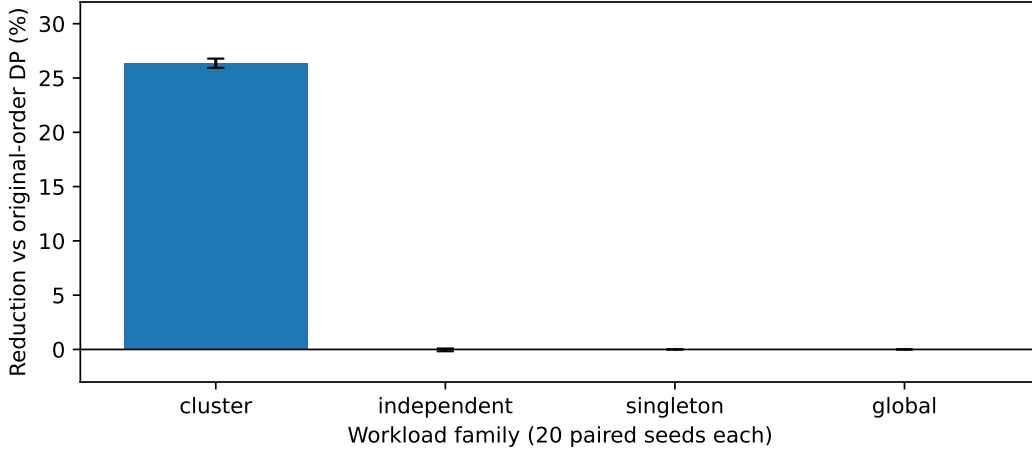


Figure 3: Archived v3. Ordering improves the structured shuffled case, not every workload. The zero line is the unchanged original-order compiler. The figure shows the same data as Table 5.

On shuffled clusters, the selected learned order reduces cost by 26.38% relative to original-order DP and 28.46% relative to fixed8. The independent control has a small negative point estimate whose interval crosses zero. Under singleton updates, the original-order compiler already captures the relevant per-check skew, and the selected proposals tie it. Global refreshes also tie. Thus the result is not “reordering always helps”; it is recovery of avoidable serialization cost when the registry is misaligned with correlated completion groups.

Construction cost and scale. With single-threaded BLAS, fitting all three proposals on 128 calibration episodes took 0.075, 0.356, 1.449, 3.637, and 18.435 seconds at $n = 32, 64, 128, 256, 512$, respectively. These are single construction measurements, not averaged throughput estimates. All produced packet maxima were 3,072 bytes. The cubic compiler should be run at controlled decision points, not after every small edit. The experiment establishes bounded packet size up to 512 *checks*, not 512 simultaneous LLM workers or constant total computation.

G.3 RQ2: Drift, switching costs, and competing policies

Four calibration regimes are fixed before evaluation: original clusters, permuted clusters, independent activity, and global activity. Their compiled layouts form a small frozen catalogue, deduplicated by manifest hash. The “frozen” comparator is chosen using original-cluster calibration. Long shift consists of four 512-event phases: original clusters, permuted clusters, independent activity, and global activity. Rapid shift alternates original/permuted clusters every eight events. Stationary and global streams are controls.

These are persistent refresh traces: per-tick costs do not include a new candidate’s unconditional full construction. Every actual topology change does include canonical manifest serialization plus a full cold reconstruction. The event’s wave is observed only after choosing the current layout. No future phase boundary is supplied to the selector. All catalogue losses are recomputed from that same wave; the measured

Table 6: Archived v3. Mean seed-level total charged cost divided by frozen-layout cost; lower is better. All migration bytes are included. CS: coupled share; IS: independent share; C0: zero-share coupling; PW: priced window; GU: unpriced greedy; oracle: clairvoyant switching comparator.

Stream	CS	IS	C0	PW	GU	Oracle
stationary	1.079	1.410	1.006	1.000	1.015	1.000
long shift	0.845	1.371	1.006	0.833	1.015	0.817
rapid shift	1.165	2.185	1.108	1.007	0.920	0.791
global	1.000	1.755	1.000	1.000	1.000	1.000

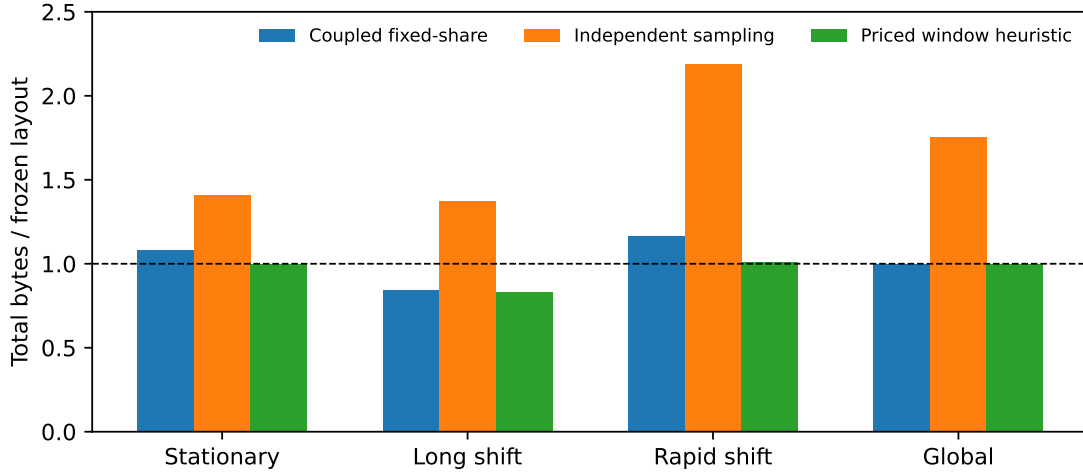


Figure 4: Archived v3. Total service plus migration cost. The dashed line is frozen-layout cost. Reduced switching matters, but the coupled policy still loses under stationary and rapid-shift conditions.

scoring and controller CPU time are stored separately.

We compare coupled fixed-share with independent sampling of the same weights, zero-share coupling, a deterministic 64-event priced-window heuristic, and a last-event greedy rule that ignores switching price when deciding but is still charged its realized migrations. A clairvoyant switching-cost dynamic program is labeled an offline comparator, not a deployable policy.

Coupled share achieves a long-shift ratio of 0.845, with 95% interval [0.834, 0.856], equivalent to a mean 15.52% reduction. Independent sampling loses enough migration bytes to exceed the frozen comparator, despite using the same evolving marginal weights. This validates the importance of temporal coupling, not a claim of a new weighted-majority method.

The negative results are consequential. Coupled share regresses by 7.88% on stationary streams and 16.45% on rapidly alternating streams; its mean switch counts are 4.55 and 111.65, respectively. The simple priced-window rule is cheaper in both these settings and in long shift. The theorem-backed policy is therefore not the empirical winner. Its value is an explicit, auditable service/migration accounting and a guarantee under stated assumptions, not universal dominance. The global control exhibits no cost differences among layouts, so maximal coupling makes zero switches, whereas independent draws incur avoidable installation cost.

G.4 RQ3: Existing source projects and fresh executions

The retained projects are NetworkX, Toolz, and the Future Code runtime, with scoped registries of 170, 148, and 138 checks. The same v2 intervention-conditioned execution ranker is used for every layout; only certificate display order changes. Training uses complete recorded mutation outcomes. A separate development cohort selects among original, spectral, and cluster layouts; clustering is selected for all three

Table 7: Archived v3. Retrospective replay on v2’s 33 published confirmation cases. Values are aggregate byte reductions in percent, including per-candidate initialization. This is not newly executed or untouched v3 generalization evidence.

Project	Checks	Cases	vs original DP	vs fixed8	vs shuffled DP
networkx	170	9	40.03	44.72	47.77
toolz	148	6	9.66	14.66	24.07
future	138	18	28.19	37.52	42.22

Table 8: Archived v3. Fresh paired executions, four candidates per project and mode. Bytes are actual emitted streams; seconds are summed outer-wrapper durations, including snapshotting and report writing. Check execution order is identical within each pair.

Project	Certificate KiB		Reduction (%)	Outer duration (s)	
	fixed8	learned		fixed8	learned
networkx	1468.9	644.3	56.14	18.35	18.51
toolz	494.5	412.2	16.63	9.87	9.88
future	689.9	447.4	35.15	11.47	11.39

projects. A deterministically shuffled-registry DP exposes sensitivity to an arbitrary order.

To check replay mechanics against actual execution, we freeze the first four lexicographically ordered confirmation IDs per project, independently of the new savings, then execute each candidate with fixed8 and the validation-selected layout. Mode order alternates by case. The exact original source hashes, mutation spans, replacement text, resulting hashes, orders, reports, and packet streams are retained. This deterministic small cohort is not a representative sample of all defects; some cases share source locations or produce unchanged test outcomes.

All 24 fresh candidate decisions match their recorded full-suite oracle verdicts. The three trusted baselines and three scripted restorations pass. The two layouts execute the same number of checks in each pair, and the independent auditor reproduces the measured packet bytes from the actual completion sequence. Across the 12 pairs, aggregate wrapper time is 39.69 seconds for fixed8 and 39.77 seconds for learned layouts. This is essentially unchanged, not a wall-clock improvement corresponding to the byte reductions. Subprocess startup, imports, snapshot acquisition, and tests dominate this small execution setting.

G.5 RQ4: Actual harness integration and I/O behavior

The new fixture contains an idempotent SQLite ledger, bounded HTTP JSON transport, and an integration service. Its 40 checks exercise disk-backed transactions, duplicate event handling, conflicting idempotency keys, rejected writes, actual loopback HTTP requests, malformed and oversized responses, HTTP errors, and end-to-end ingestion. The tests use real SQLite and socket I/O; the failures and repaired source content are scripted. Network delays, remote services, and nondeterministic production systems are not represented.

A real Future Code supervisor receives a root coordinator task that delegates to two children with disjoint write scopes. Each child reads its module, submits an incorrect implementation, receives an executable rejection, then submits the known correct implementation. The parent verifies integration. A local HTTP server supplies the scripted model actions and injects one HTTP 503 before a successful retry. The model-response script is identical across gate modes. Provider usage is absent and recorded as unknown.

Three repetitions rotate mode order among direct pytest, strict fixed8, and strict learned-layout gates. The strict variants use trusted baseline contracts outside candidate scopes and the actual bridge subprocess. Their layout calibration is an explicit predeployment completion-wave fixture, not training on evaluated episode outcomes. Setup/contract fitting is recorded separately from active episode time. The small fixture

Table 9: Archived v3. Nine actual harness episodes, three per mode. Duration runs from supervisor startup through completion and shutdown. Prompt bytes include retries and all worker calls, not hosted tokens. Certificate bytes are not applicable for direct pytest.

Mode	Runs	Median s	Range s	Mean cert. KiB	Mean prompt KiB
direct	3	12.98	12.55–13.50	–	71.39
fixed8	3	18.11	17.45–18.29	95.62	70.23
learned	3	18.12	17.12–18.13	95.62	70.48

is intentionally capable of showing no layout benefit.

All nine episodes complete the three-task graph with an observed peak of two active workers, reject at least the two intentionally incorrect candidates, and recover from the injected outage. All final source modules equal the scripted correct inputs. Strict modes produce 97,920 certificate bytes per episode and tie each other. Their median active time is about 18.1 seconds versus 13.0 seconds for direct gates. This roughly five-second premium buys the additional source/contract checks and evidence materialization; it is not an optimization speedup.

The maximum observed aggregate message-content size per HTTP request is 6,902 UTF-8 bytes, below the runtime’s 16,384-byte input cap. Mean total context-construction time is 6.3–7.2 milliseconds per episode; total prompts are about 70–71 KiB. These measurements do not imply a statistically established prompt reduction, billing reduction, or improvement in model cognition. Only two simultaneous workers are measured here.

G.6 Software verification and independent audit

The full prototype suite passes 187 tests, including 43 new v3 tests beyond v2’s 144. Separately, the vendored Future Code runtime passes all 263 of its own tests. New cases cover invalid and duplicate permutations, disconnected affinity graphs, degenerate spectral choices, exhaustive small-order comparisons, pairwise information loss, coupled marginal distributions, checkpoint continuation, choice-before-observation enforcement, priced migration, canonical-ID remapping, stale tickets, delayed evidence, contradictory records, bridge subprocesses, and independent-auditor tamper rejection.

The audit imports neither the compiler nor producer cost helpers. It reconstructs all 80 ordering workloads, all 80 online streams (163,840 events), the 33 archived replay cases, 24 fresh paired runs, and nine harness episode records. It also checks 63 actual binary packet streams containing 2,305 packets and independently recomputes 32 reported bootstrap comparisons. A label mismatch that initially left an empty cohort in the draft summary was detected and corrected; summaries now reject missing cohorts instead of treating absence as zero cost. No erroneous draft statistic is included in the paper. These checks are independent implementation paths, not an external replication or security certification.

H Historical V4 Delivery and Compiler Evidence

The observations and implementation recommendations in this appendix are historical v4 records. The v5 practical path excludes block DP; its current recommendation and new measurements are in Section 9.

H.1 Frozen questions and provenance

The v4 contract was written before its new measurements. It fixes all 24 packet streams from v3’s 12 matched source-checker pairs, four encodings, two timed encodings, three link settings and two repetitions. It separately fixes 18 new synthetic compiler workloads (three seeds, two families, three sizes). The data categories remain distinct: v3 contains executed check outcomes; v4 re-exports those bytes and constructs new synthetic compiler traces. Re-exporting a packet is not another successful code repair.

Table 10: V4 byte reductions (percent) versus the matched fixed8 layout, including application framing. Four archived candidates per project; positive is better. Every column preserves the same intermediate packet records.

Project	Padded	Canonical	Per-packet zip	Epoch zip
networkx	56.12	56.25	55.67	47.92
toolz	16.62	16.73	15.64	5.80
future	35.13	35.28	34.53	23.60

Table 11: Historical v4 controlled TCP exports. Seconds are total export time for 12 candidate streams, averaged over two repetitions. The study is not a new run of their checkers or an end-to-end agent evaluation.

Receiver pacing	Format	Fixed (s)	Learned (s)	Reduction (%)
64 KiB/s	Canonical	33.499	18.954	43.42
64 KiB/s	Epoch-compressed	6.624	4.413	33.38
1 MiB/s	Canonical	2.322	1.318	43.26
1 MiB/s	Epoch-compressed	0.673	0.429	36.26
Unpaced	Canonical	0.243	0.138	43.49
Unpaced	Epoch-compressed	0.283	0.156	44.91

For transport, each trial opens a real loopback TCP connection. The receiver consumes chunks under an application-rate limiter, decodes the received archive, compares its SHA-256 to the trusted experimental reference, and returns an acknowledgment. Timers include sender encoding, connection and transfer, receiver decoding and acknowledgment. Application byte counts include the codec envelope, packet framing, the eight-byte outer length and 32-byte acknowledgment. They exclude TCP/IP/TLS overhead. No receiver-rate policy is represented as a measured physical WAN. Tests are not reexecuted during these transfers.

The timed formats are unpadded canonical JSON and whole-epoch DEFLATE at level 6; padded and independently compressed packets are additional size controls. All 288 trials are retained. Order is randomized by the frozen seed. Tables aggregate the 12 predetermined exports and average the two repetitions; they are descriptive instrument measurements, not population confidence intervals for arbitrary projects. Initial layout compilation is outside the export timer and reported separately.

H.2 RQ1: Does the saving survive compression?

After whole-epoch compression, the reductions are 47.92% for NetworkX, 5.80% for Toolz and 23.60% for Future Code. In particular, Toolz’s 16.62% padded-wire saving falls to 5.80%. Compression and batching are therefore substantive baselines, not negligible framing details. The fixed layout with compression is much cheaper than an uncompressed learned stream; claims must compare like with like.

The compressed payload is a post-verification archive. Independent packet compression preserves per-packet streaming but yields different savings. Neither codec is optimized by the padded-byte DP, and no universal preservation of the learned layout’s ranking is assumed. A flat final-status ledger is also much smaller in these cases, but omits intermediate aggregation and trace commitments. Its size is recorded as an alternative for a trusted central consumer, not as a substitute for a portable proof or the same diagnostic service.

H.3 RQ2: Does smaller evidence improve actual export time?

At 64 KiB/s, canonical export takes 33.499 versus 18.954 seconds for the 12-stream cohort (43.42% reduction). With epoch compression the same totals are 6.624 and 4.413 seconds (33.38%). The approximately 2.21-second compressed saving is *across twelve exports*, not per candidate. All decoded packet counts and payload digests match the archived inputs.

Table 12: Historical v4 compiler measurements. Times are medians of three seeds in seconds. Gap is the mean held-out blocked cost relative to the same-order exact cost; positive is worse. No new asymptotic claim is inferred from timing.

Checks	Family	Order	Exact DP	Block DP	Gap (%)
128	cluster	0.017	0.148	0.033	+1.62
128	independent	0.018	0.161	0.035	+3.68
512	cluster	0.534	3.465	0.132	+3.40
512	independent	0.581	3.432	0.136	+0.40
1024	cluster	2.839	—	0.260	—
1024	independent	3.198	—	0.278	—

Unpaced and 1 MiB/s runs also improve in this controlled setting; encoding and decoding fewer records contributes when pacing no longer dominates. The timings are not predictions for TLS, congested networks, remote storage, a concurrent multi-link tree or a deployment that hides export behind checking. Compression itself is the larger immediate optimization here. Adding an expensive strict wrapper solely to obtain a small export saving would not follow from these results.

The historical checker and harness negative controls are unchanged: fixed and learned source wrappers total 39.69 and 39.77 seconds; the nine scripted harness episodes show an approximately five-second strict-gate premium over direct pytest. Complete details remain in Appendix G. Combining those historical times with new loopback times and calling the sum a measured end-to-end speedup would be invalid.

H.4 RQ3: Which supervisor context actually gets smaller?

The root-status representation is exactly identical within all 12 candidate pairs and occupies 593–594 UTF-8 bytes. Its *layout-induced* reduction is zero, consistent with Equation (14). Fixed-size root reporting is already sufficient for ordinary status display. We do not claim that an optimizer is necessary to achieve it.

The exhaustive diagnostic path differs. With a 16,384-byte per-request cap and whole-packet packing, NetworkX uses 82 versus 36 requests across four candidates, Toolz 27 versus 25, and Future Code 39 versus 26. Overall, the cohort requires 148 versus 87 diagnostic pages (41.22% fewer), while every page respects the same cap. These are exact requests needed by the implemented exhaustive reader, not observed tool-use choices by an LLM. Selective retrieval could use far fewer pages, and a root-only supervisor uses none of them.

No tokenizer was installed in this execution environment; attempts to obtain its package/vocabulary failed because network resolution was unavailable. We therefore report exact input bytes against the harness’s actual byte-admission limit and leave tokenizer counts null. The optional hook invokes a real tokenizer [19] when available; it never estimates tokens from bytes. No claim is made about model understanding, billing or the token-window fit of these diagnostics.

H.5 RQ4: Can compilation cost be bounded usefully?

The new compiler study fixes a training-learned average-linkage order, 128 training epochs, 256 held-out epochs, and block size 32. The two families are shuffled clustered and independent completions. Exact and blocked methods share the same order and full-wave objective; order-learning time is measured separately. At 1,024 checks only blocked compilation is performed; an unmeasured exact timing or gap is shown as missing, not extrapolated.

At 512 checks, blocked microtree compilation is roughly 25–26 times faster than the same-order exact DP, with mean held-out penalties of 3.40% (clustered) and 0.40% (independent). The entire pipeline does not accelerate by that factor: learning the order still costs about 0.53–0.58 seconds. At 1,024 checks, microtree fitting is about 0.26–0.28 seconds and ordering about 2.84–3.20 seconds. The old 18.435-second v3 measurement fitted a *three-proposal catalogue*; it was not a directly comparable single-DP timing on this

runtime. Restricted compilation and order reuse are practical options, not a claim that global optimization is now linear or free.

A simpler baseline is important: the blocked trees use 1.54–5.34% more held-out bytes than a balanced eight-ary tree on the same learned order across these six settings. The block compiler is therefore an optional bounded-cost candidate generator, not the default winner. Selection must retain the incumbent and a simple fixed-fanout candidate; faster fitting alone does not justify adopting a worse layout.

H.6 Online policy and software validation

The deterministic window is promoted from comparator to primary runtime API without parameter retuning. Across all 80 archived online streams (163,840 events), the new API reproduces every prior baseline action and byte-cost total, including periodic checkpoint/resume. This establishes compatibility, not fresh generalization evidence. In v3 its charged-cost ratios to a frozen layout were 1.000 (stationary), 0.833 (long shift), 1.007 (rapid shift) and 1.000 (global). A last-event greedy rule performs better on the rapid-shift family, so the window is not declared universally best. The weaker fixed-share results and theoretical analysis remain visible in the appendix.

The v4 release passed 253 tests, including 66 new tests beyond v3’s 187. That release separately passed 263 unmodified Future Code tests. New tests check constrained compilation, exact codec round trips, decompression bounds, malformed partitions, root projection, context-budget overflow, controller sequencing/checkpoints, and independent-auditor rejection of altered measurements. A duplicate test-module basename initially prevented collection; it was renamed, the failure log retained, and the complete suite rerun.

The independent audit imports no producer optimizer or codec. It reconstructs packet framing, compression sizes, page packing and compiler costs from saved records; checks all 288 transfer ledgers, 18 compiler workloads and 80 historical online replays; and verifies identical paired root projections and independently recomputes the reported numerical summaries. This is an independent implementation path, not third-party replication or a security certification.